RoomsScreen

La pantalla que muestra qué habitaciones están libres, agrupadas por tipo.

Rony Valdelamar
Introducción y estructura

Jair Sánchez Pérez
Widgets de estructura

Victor Rocha Ramos
Widgets visuales y datos

Aníbal Meza Anaya
Lógica, navegación y mejoras

Cómo nos repartimos

01 — 03 · 13

Rony

Qué problema resuelve, cómo está construida, cómo se ve y los conceptos clave.

04 — 05 · 14

Jair

Scaffold, AppBar, ListView, Card, ExpansionTile, ListTile y las preguntas 1 a 5.

06 — 08 · 15

Victor

Text, Icon, Chip, tipos del modelo, entrada de datos y las preguntas 6 a 10.

09 — 12 · 16

Aníbal

Métodos, Hotel, navegación, resultados, mejoras y la prueba práctica.

Expone: Rony

¿Qué problema resuelve?

El recepcionista necesita saber en segundos **qué habitaciones puede ofrecer** antes de hacer una reserva.

REQUISITO

RF03

Consultar las habitaciones disponibles agrupadas por tipo.

QUÉ MUESTRA

Por tipo

Sencilla, Doble y Suite, con la cantidad libre, el número y la capacidad.

NATURALEZA

Solo lectura

Consulta información; no cambia nada en el hotel.

Expone: Rony

¿Cómo está construida?

- Dos clases, ambas `StatelessWidget`.
- `RoomsScreen`: la pantalla completa.
- `_EstadoChip`: etiqueta de color con el estado. El `_` la hace privada al archivo.
- Es Stateless porque no guarda datos propios: cada vez que se abre, le vuelve a preguntar al `Hotel`.

```
Scaffold
├─ appBar: AppBar → Text('Habitaciones disponibles')
└─ body: ListView
   └─ por cada tipo → Card
      └─ ExpansionTile
         ├─ title: 'Doble (3 disponible(s))'
         └─ children: ListTile × habitación
            ├─ leading   Icon puerta
            ├─ title     'Habitación 104'
            ├─ subtitle  'Capacidad: 2'
            └─ trailing  _EstadoChip
```

Expone: Rony

Así se ve

Al iniciar la app, `Hotel` crea **3 habitaciones por tipo**. Todas empiezan disponibles.

101–103

Sencilla · 1 persona

104–106

Doble · 2 personas

107–109

Suite · 4 personas



Ejemplo: la 104 ya fue reservada y las suites están todas ocupadas

Expone: Jair

Estructura de la pantalla

Scaffold

El esqueleto de la pantalla

`appBar`
Pone la barra superior con el título.

`body`
El contenido principal: aquí, el ListView.

`backgroundColor` no se usa
Cambia el color de fondo de la pantalla.

AppBar

La barra superior

`title`
Texto de la barra: "Habitaciones disponibles".

`centerTitle` no se usa
Si es true, centra el título.

`actions` no se usa
Botones a la derecha, como uno de refrescar.

ListView

Lista con scroll

`padding`
`EdgeInsets.all(16)`: 16 px de espacio por cada lado.

`children`
Los elementos: una Card por cada tipo de habitación.

`scrollDirection` no se usa
Scroll vertical (por defecto) u horizontal.

Las propiedades marcadas "no se usa" existen en Flutter; las añadimos para completar tres.

Expone: Jair

Tarjetas y filas

Card

Tarjeta con bordes redondeados

`margin`
`only(bottom: 16)`: separa cada tarjeta de la siguiente.

`child`
Lo que va dentro: el ExpansionTile.

`elevation` no se usa
Qué tan marcada es la sombra.

ExpansionTile

Sección que se abre y se cierra

`initiallyExpanded`
`true`: aparece abierta, así se ve todo sin tocar nada.

`title`
Encabezado: tipo y cantidad disponible.

`children`
Las filas de habitaciones, o el aviso de que no hay.

ListTile

Fila estándar de una lista

`leading`
A la izquierda: el ícono de puerta.

`title · subtitle`
"Habitación 104" y "Capacidad: 2 persona(s)".

`trailing`
A la derecha: el chip de estado.

Expone: Victor

Texto, íconos y estado

Text

Muestra texto

`data`
El texto, armado con interpolación: `'$tipo (${n} ...)'`.

`style`
`TextStyle(fontWeight: bold)`: título en negrita.

`textAlign` no se usa
Alineación: izquierda, centro o derecha.

Icon

Ícono de Material Design

`icon`
`door_front_door` o `info_outline`.

`size` no se usa
Tamaño del ícono en píxeles.

`color` no se usa
Color del ícono.

Chip

Etiqueta tipo píldora

`label`
"Disponible", "Reservada" u "Ocupada".

`backgroundColor`
El color del estado con `withOpacity(0.15)`: un tono suave.

`labelStyle`
El texto en `shade700`, el tono oscuro del color.

El color sale de un switch: disponible → verde · reservada → naranja · ocupada → rojo

Expone: Victor

Los datos detrás de la pantalla

Habitacion

Clase del modelo

`numero`
101, 102… Se muestra en cada fila.

`estado`
Lo único que cambia. Decide si aparece en la lista.

`capacidad`
Getter: se calcula desde el tipo, no se guarda.

TipoHabitacion

enum: sencilla, doble, suite

`values`
Los 3 tipos. Se recorren para crear una tarjeta por tipo.

`capacidadMaxima`
1, 2 y 4 personas.

`toString()`
Texto bonito: "Sencilla", "Doble", "Suite".

EstadoHabitacion

enum del ciclo de vida

`disponible`
Libre. Es el único estado que se lista aquí.

`reservada · ocupada`
Después de reservar o de hacer check-in.

`toString()`
El texto que se ve en el chip.

Map<TipoHabitacion, List<Habitacion>> une todo: a cada tipo le corresponde su lista

Expone: Victor

Recibe, captura y valida

¿QUÉ RECIBE?

Un objeto Hotel

Lo recibe por el constructor con `required`. Es el **mismo** Hotel creado en `main.dart` y compartido por todas las pantallas.

¿CÓMO CAPTURA?

No captura

No tiene formularios ni TextField. El usuario solo abre o cierra las secciones, hace scroll y vuelve atrás.

¿CÓMO VALIDA?

Lo hace Hotel

Hotel filtra con `estaDisponible`. La pantalla solo revisa `isEmpty` para mostrar un aviso si un tipo no tiene habitaciones.

Regla del proyecto: las pantallas muestran; la clase Hotel aplica las reglas de negocio

Expone: Aníbal

¿Qué pasa al abrirla?

01

build()

Flutter construye la pantalla.

02

consultarDisponibles-Agrupadas()

Se le piden los datos al Hotel.

03

consultarDisponibles()

Filtra con `where(estaDisponible)`.

04

Agrupa

Cada habitación va a la lista de su tipo dentro del Map.

05

map().toList()

Convierte cada tipo en una Card y cada habitación en un ListTile.

Además: el getter `capacidad`, el `switch` de `_EstadoChip.build()` para el color y el `toString()` de los enums para los textos.

Expone: Aníbal

Hotel y navegación

COMUNICACIÓN CON HOTEL

- Llama directamente a `hotel.consultarDisponiblesAgrupadas()`.
- Es de **una sola vía** y de **solo lectura**: pide datos y nunca los cambia.
- Toda la lógica queda en un solo lugar.

NAVEGACIÓN

- **Entrada:** desde Home con `Navigator.push(MaterialPageRoute(...))`.
- **Salida:** la flecha de atrás del AppBar hace `pop` y vuelve al menú.
- No lleva a ninguna otra pantalla.

Expone: Aníbal

¿Y si algo sale mal?

CASO NORMAL

Hay habitaciones

Se ve cada tipo con su cantidad, y cada habitación con su capacidad y un chip verde.

LISTA VACÍA

Tipo sin cupo

Aparece "(0 disponible(s))" y la fila "Sin habitaciones disponibles". Es un aviso, no un error.

ERRORES

Sin excepciones

No tiene try/catch ni SnackBar: solo lee datos. Los mensajes de error están en las pantallas que modifican el hotel.

Si se reserva una habitación, desaparece de esta lista al volver a abrirla

Expone: Aníbal

¿Qué mejoraríamos?

01

Ver todos los estados

El chip ya soporta los colores naranja y rojo. Mostrar todo el hotel, no solo lo libre.

02

Reservar desde aquí

`onTap` en el ListTile para abrir ReserveScreen con el número ya puesto.

03

Refrescar

Pasar a StatefulWidget con un botón en `actions` o un RefreshIndicator.

04

Total en el AppBar

Mostrar "Disponibles: 9" en el título.

05

Filtrar por personas

Mostrar solo las habitaciones donde cabe el grupo.

06

Código actualizado

`withOpacity` está obsoleto: se cambia por `withValues(alpha:)`.

Expone: Rony

Los conceptos en una línea

`StatelessWidget`
Widget sin estado propio; se redibuja con los datos que recibe.

`Map`
Diccionario clave → valor. Aquí: tipo → lista de habitaciones.

`List`
Colección ordenada. Cada tipo tiene su `List<Habitacion>`.

`enum`
Valores fijos y conocidos: `TipoHabitacion` y `EstadoHabitacion`.

`map()`
Transforma cada elemento: un tipo en una Card, una habitación en un ListTile.

`ListView`
Lista con scroll que contiene las tarjetas.

`ExpansionTile`
Sección que se abre y se cierra al tocarla.

`ListTile`
Fila con ícono, título, subtítulo y algo a la derecha.

`Chip`
Etiqueta pequeña: muestra el estado con color.

`switch`
Elige el color según el estado: verde, naranja o rojo.

`Theme.of(context)`
Lee el tema global de la app (colores, tipografía). Ojo: RoomsScreen no lo usa; los colores del chip son fijos. Sí aparece en HomeScreen para pintar los íconos con el color primario.

Expone: Jair

Preguntas 1 a 5

01 **¿Por qué RoomsScreen es StatelessWidget?**
Porque no guarda datos propios que cambien mientras está abierta. Los datos viven en `Hotel` y se leen de nuevo en cada `build()`.

02 **¿Qué devuelve consultarDisponiblesAgrupadas()?**
Un `Map<TipoHabitacion, List<Habitacion>>` con las habitaciones disponibles separadas por tipo. Siempre trae los 3 tipos, aunque alguno esté vacío.

03 **¿Qué representa Map<TipoHabitacion, List<Habitacion>>?**
Un diccionario: la **clave** es el tipo (Sencilla, Doble, Suite) y el **valor** es la lista de habitaciones libres de ese tipo.

04 **¿Por qué se utiliza TipoHabitacion.values?**
Devuelve todos los valores del enum. Así se crea una tarjeta por cada tipo y, si mañana se agrega uno nuevo, aparece solo.

05 **¿Qué función cumple map()?**
Transforma cada elemento de una lista en otro: cada tipo en una `Card` y cada habitación en un `ListTile`. `.toList()` lo convierte en lista para `children`.

Expone: Victor

Preguntas 6 a 10

06 **¿Qué hace ExpansionTile?**
Crea una sección desplegable: muestra un título y, al tocarlo, abre o cierra sus `children` (las habitaciones de ese tipo).

07 **¿Qué representa _EstadoChip?**
Un widget propio y privado (por el `_`) que dibuja una etiqueta de color con el estado de la habitación.

08 **¿Por qué _EstadoChip recibe estado?**
Porque con ese dato decide, mediante un `switch`, el color (verde, naranja, rojo) y el texto. Así es reutilizable para cualquier habitación.

09 **¿Qué pasa si una habitación cambia de disponible a reservada?**
Deja de pasar el filtro `estaDisponible`. La próxima vez que se abra la pantalla ya no aparece y el contador de su tipo baja en uno.

10 **¿Por qué no modifica directamente la lista de habitaciones?**
Porque su trabajo es solo mostrar. Las reglas de negocio y los cambios de estado son responsabilidad de `Hotel`; así la lógica queda en un solo lugar.

Expone: Aníbal

Habitaciones inicialmente contraídas

ANTES · ROOMS_SCREEN.DART, LÍNEA 40

```
child: ExpansionTile(
  initiallyExpanded: true,
  title: Text(...),
```

DESPUÉS

```
child: ExpansionTile(
  initiallyExpanded: false,
  title: Text(...),
```

PROPIEDAD

initiallyExpanded

Define si la sección empieza abierta (`true`) o cerrada (`false`).

RESULTADO

Solo títulos

Se ven "Sencilla (3)", "Doble (3)"... y las habitaciones aparecen al tocar cada tipo.

ALTERNATIVA

Borrar la línea

Su valor por defecto ya es `false` , así que quitarla da el mismo resultado.

Guardar y hacer hot reload (r) para ver el cambio